TFLBot: A Helping Hand Around Underground Transport Networks

Tshepo Romeo Nkutlwang (02196902), Sandro Enukidze (02207144), Dinushan Edmond Camilus (02256620), Isobel Owens (02260259), Dhruv Varsani (02313221), Adeola Olawoye (02271386), Sophie Jayson (02254802), Akin Falase (02199338), Dylan Winters (02122370), Arundhathi Pasquereau (02207669)
Imperial College London

***Abstract*—Navigating large metropolitan public transport systems can be challenging, particularly for first-time users and passengers with accessibility needs. Complex station layouts, static signage, and reliance on staff availability can increase cognitive load and travel-related anxiety. This project explores the feasibility of TFLBot, a mobile assistive robot designed to support passengers in underground transport environments through autonomous navigation guidance and accessible, multimodal interaction. The system combines autonomous mobility with speech, visual feedback, and British Sign Language (BSL) to provide informative and inclusive assistance. This report presents the system design, implementation decisions, and evaluation approach for assessing the effectiveness of robotic guidance in complex public transport settings.**

I. Introduction

This project investigates the effectiveness of a mobile assistive robot for passenger guidance in underground transport environments. Specifically, it evaluates two assistance modes: (i) a mobile, embodied guidance mode, in which the robot physically escorts users and communicates through verbal and non-verbal cues, and (ii) a stationary, instruction-based mode, in which guidance is provided from a fixed location.

The TFLBot is designed to operate autonomously within metropolitan stations. Users approach the robot and ask for help with navigation or information. The system then integrates autonomous mobility, perception, and multimodal interaction to guide users or answer questions. TFLBot is designed to provide guidance using both modes for testing.

Underground transport networks are distinctly challenging to navigate compared to other unfamiliar environments such as hotels or airports. Stations are often multi-level and complex, with dense foot traffic, lower lighting, and unreliable network signals. Noise, crowding, complicated maps and sparse signs can also make instructions harder to follow. These characteristics make underground transports systems a particularly suitable setting for evaluating whether embodied guidance can improve navigation efficiency.

One important focus is the use of non-verbal and embodied communication to reduce cognitive demands associated with navigation. By shifting the burden from interpreting verbal instructions or maps, the system aims to improve navigation efficiency while lowering user cognitive load.

This report presents the design and implementation of the TFLBot system and describes an experimental methodology that compares the two interaction modes. Performance and user experience are evaluated using objective navigation metrics and subjective workload and satisfaction measures. The findings offer insights into the effectiveness and practicality of robotic guidance in underground public transport environments, and highlight its potential as a viable solution for assisting passengers.

II. Hypothesis

H1: When navigating a station, the presence of robotic guidance will improve user experience, reduce perceived workload, and reduce navigation time compared to no guidance.

H2: A mobile assistive robot that provides physical guidance with verbal and visual cues will further improve user experience, reduce perceived workload, and reduce navigation time compared to a stationary robot that only provides verbal directions.

The hypotheses of this study are that the introduction of robotic guidance will improve user experience and navigation performance compared to no guidance, and that a mobile assistive robot providing physical guidance supported by verbal and visual cues will further enhance these outcomes compared with a stationary robot providing only directions.

These hypotheses are based on the premise that both the presence of guidance and the modality through which it is delivered influence effectiveness in complex environments such as train stations. While a stationary system provides users with verbal instructions, a mobile robot is capable of accompanying users through the station while reinforcing directions through physical movement, hand gestures, and visual indicators.

By providing multimodal guidance during navigation, the mobile robot is expected to improve clarity of directions, increase user confidence, and reduce perceived workload. Additionally, the presence of any robotic guidance is anticipated to offer measurable benefits over no guidance, particularly for individuals unfamiliar with the station layout or navigating crowded environments.

III. Background Research

A. Robotic Assistance in Public Transport Environments

Robotic systems have been deployed in public transport environments to support passenger assistance, primarily through information provision and navigation support. One example is the Pepper robot, which has been trialled in metro stations and

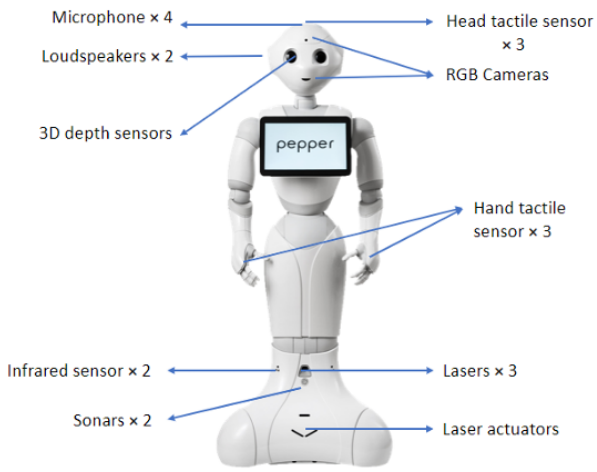

Fig. 1. Pepper Bot

railway terminals to provide directions and service information via speech and an integrated display [1]. These deployments typically position the robot as an information point rather than an active guide.

More recently, commercial platforms such as the LG Airport Guide Robot and LG CLOi GuideBot have been introduced in large transport hubs, where they provide mobile guidance by escorting users to destinations while delivering navigational information [2], [3]. As combined guides and information points, they offer useful reference points for improving user experience, including features such as multilingual support.

B. Mobile Escort and Wayfinding Robots

Beyond transport-specific deployments, a broader class of mobile robots has been developed for escort and wayfinding tasks in complex public environments. The SPENCER robot demonstrated that embodied guidance can reduce navigation ambiguity by allowing users to follow the robot directly rather than interpret verbal directions [4].

Commercial platforms such as the temi personal robot have also been adapted for escort-style guidance in airports and large venues [5]. However, these systems are typically designed for open environments and provide limited accessibility-focused interaction.

C. Social and Concierge-Oriented Service Robots

Another class of robotic systems prioritises social engagement, with navigation treated as a secondary function. Robots such as Mirokaï emphasise expressive motion and conversational interaction to improve approachability in public spaces [6]. Similarly, Pepper is frequently deployed in concierge roles rather than as an active guide [7]. Though in these systems, mobility primarily enhances presence rather than navigation capability, they provide a useful reference for designing and improving communication.

D. Accessibility and Interaction Limitations

Existing systems remain largely focused on speech- and screen-based interaction. While embodied guidance through physical motion has been shown to reduce navigation ambiguity, non-verbal communication cues such as gesture are often limited or treated as secondary interaction elements [4], [8]. Support for accessibility-focused interaction, including sign language or adaptive multimodal feedback, remains rare in deployed transport robots.

Furthermore, evaluations of robot guides often prioritise technical performance and navigation success over user-centred measures such as cognitive load or satisfaction, limiting insight into the benefits of different guidance modes.

IV. System Design

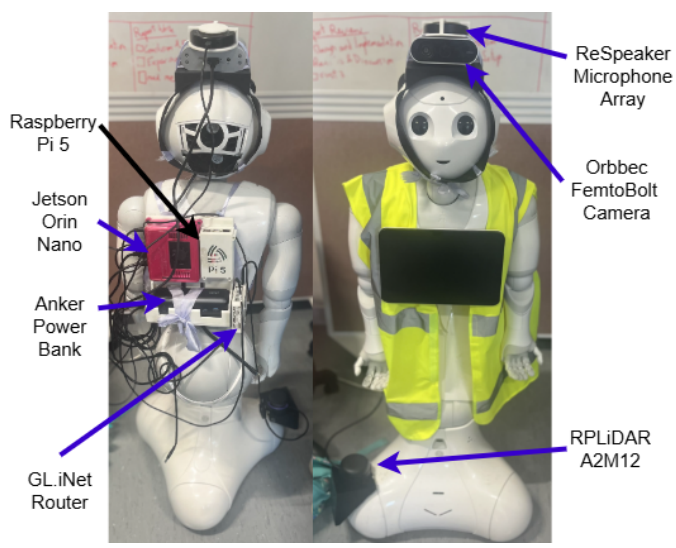

Fig. 2. Upgraded Pepper

The overall system design comprises of multiple subcomponents (as seen in Figures 2 and 3) as a result of the Pepper's limited sensory and onboard processing capabilities, providing greater flexibility in the control architecture, as each module can be more directly interfaced with compared to using the Pepper itself. Note the inclusion of the Raspberry Pi 5 [9] and a Jetson Orin Nano [10] to allow for multiple simultaneous commands and image processing of data from the Femto Bolt Camera. We also make use of Gemini [11] to process speech received from the microphone, and also real time TFL Status updates. All of these additional components are to be powered by the Anker 165W battery [12]. Additionally, a GL.iNet router was attached [13] to allow for communication between devices over LAN.

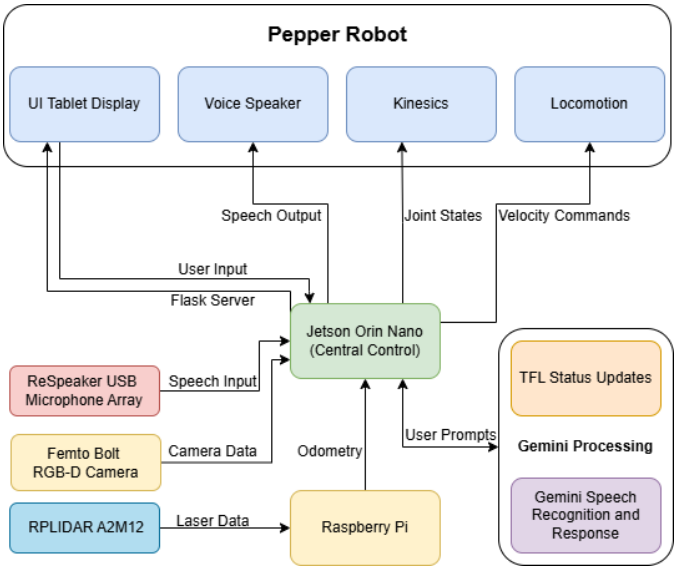

Fig. 3. System Design Diagram

The system uses a variety of ROS2 Humble packages, with the main one being the naoqi-ros2 bridge [14]. This bridge to the NaoQi library runs on the Jetson (the central controller), to allow for the processing and posting of speech, wheel, and upper body movement commands. Alongside this bridge node, we have the UI running (on the Jetson) through a Flask server embedded in a ROS2 wrapper, allowing for easy communication between the user commands and the naoqi-bridge node. The camera is interfaced with using the pyorbbecsdk [15], alongside a publisher that constantly streams BSL data to a topic, to be processed by the Flask ROS2 node.

LLM output is published to a ROS topic as a structured string, which is then processed by a task manager node subscribed to this topic. The task manager interprets the content and dispatches appropriate commands to speech, joint control, and navigation goal topics, enabling coordinated robot behaviour. The system supports multiple interaction states, including greetings, thinking behaviours, and synchronised gesture execution. It also incorporates basic failure handling mechanisms, allowing the robot to respond robustly to errors such as unrecognised input or delayed responses.

A. Softbank Pepper

The Pepper robot [16] is a 1.2-meter-tall social humanoid developed by SoftBank Robotics in 2014, engineered with 17 joints for fluid movement and a 12-hour battery life to serve in education, healthcare, and hospitality. Its hardware architecture features a sophisticated sensory suite including 4 microphones, 2 loudspeakers, 3D depth sensors, and lasers.

Empirical research, however, reveals significant technical inconsistencies. Specifically, the built-in SLAM navigation is considered unreliable due to a reported average localisation error of 0.51 meters and a blocking mapping process that prevents the robot from executing other commands until exploration is complete. While Pepper demonstrates satisfactory sound localisation and speech recognition, its facial recognition efficiency drops significantly beyond a range of 2 meters, and the robot often experiences confusion when managing multiple simultaneous speech inputs or navigating in extreme lighting conditions.

In addition to this, Pepper's onboard processing, based on an 1.9 GHz quad-core Atom processor with approximately 2 GB of usable RAM and no dedicated GPU, offers limited processing power for modern vision and perception tasks, which commonly require multi-core CPUs and 8–32 GB RAM.

B. Speech Recognition and Response

Speech recognition was implemented to enable natural spoken interaction between users and the robot within the station environment to aid in simplar communication. The system was developed in Python using the `SpeechRecognition` library, which provides a high-level interface for capturing audio from a microphone and converting it into text using external recognition services. This library was selected due to its simplicity of integration with Python, support for multiple speech recognition engines, and built-in functionality for microphone input and ambient noise calibration. These features made it well suited for rapid prototyping and experimentation within our robotics system.

Audio input is captured through a ReSpeaker USB microphone array [17]. This device provides multiple microphones and supports speaker localisation, allowing the system to estimate the direction of incoming speech. The ReSpeaker array was chosen over standard single microphones because it improves speech capture quality in noisy environments and enables directional detection of the speaking user.

Once audio is captured, the SpeechRecognition library processes the signal and sends it to Google's speech recognition service for transcription. Google's recognition service was selected because it offers high transcription accuracy and supports multiple languages, which is important in an international transport environment. Error handling was implemented to manage cases where speech cannot be understood or when external recognition services are unavailable.

After transcription, the recognised text is analysed to extract relevant navigation information. The system searches the recognised sentence for keywords corresponding to London Underground lines (e.g., *Central*, *Victoria*, *Jubilee*) and directional terms (e.g., *northbound*, *eastbound*). Identifying these keywords allows the system to recognise user queries related to specific routes and directions within the station so the robot can guide the user to the correct platforms.

The interpreted request is then passed to the Gemini AI model [11], which generates an appropriate navigation response. Gemini was selected due to its strong natural language processing capabilities and its ability to interpret complex queries that may not follow rigid command structures. This allows users to ask questions in a more natural conversational manner.

To communicate responses to the user, the robot uses the built-in speech synthesis capabilities provided by the Pepper platform through the NAOqi framework, specifically the `ALTextToSpeech` module. This approach enables real-time verbal interaction that is tightly integrated with the robot's behaviour and dialogue system. Compared to external text-to-speech libraries, Pepper's native speech system offers seamless integration with onboard sensors and control modules, ensuring low-latency responses and more coherent human–robot interaction. Additionally, it supports configurable voice parameters such as language, pitch, and speed, allowing the system to be adapted for different user needs and environments.

C. Human Detection and Sign Language Recognition

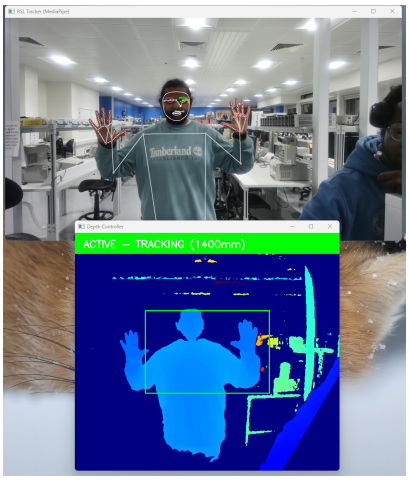

Fig. 4. Screenshot of the depth and RGB camera streams during human recognition

This submodule enables the robot to detect nearby users and interpret sign language gestures for communication. To improve perception capabilities, the system incorporates an upgraded vision setup using the Orbbec Femto Bolt camera, replacing Pepper's native depth sensing.

Upgrading the Pepper robot with an Orbbec Femto Bolt Camera [18] provides a significant leap in spatial awareness and environmental interaction by replacing the original, often unreliable depth system with 1 megapixel Time-of-Flight (ToF) technology. This camera offers a high-resolution 1024x1024 depth map and a wide 120-degree field of view, which directly addresses Pepper's known navigation flaws, such as the 0.51-meter localisation error and poor obstacle detection in complex lighting. Furthermore, the Femto Bolt integrates a 4K HDR RGB camera and a 6-axis IMU, enabling precise depth-to-colour registration and better orientation tracking. Because it is compatible with the Microsoft Azure Kinect SDK and features a built-in Human Skeleton Tracking SDK, it enables the robot to track bodies and recognise gestures with much higher reliability than the stock hardware, especially beyond the typical 2-meter visual range where the original sensors often fail.

The initial stage of this system involves human detection. An array microphone provides rough localisation data to compensate for the Femto Bolt Depth and the RGB camera's lack of a 360° field of view (FOV).

The depth camera was used to detect the presence of an object within the robot's FOV and to determine its distance. If the person is too close, the system suspends computer vision processing because only part of the hand is likely to be seen. Conversely, if a person exceeds a specified threshold, the system suspends computer vision processing because interaction intent is unlikely, to prevent unnecessary processing of background noise.

Figure 4 shows both camera streams of the depth camera and RGB behind. The depth stream contains a rectangular region at the centre that displays the window within which a person must be positioned for recognition. This was added because the more centred a potential user is, the more likely they are to be a user rather than a bystander. [19]

When in the correct range from the TFLBot, the program uses Google's MediaPipe to identify key landmarks of the hand and wrist, which are then saved in arrays for classification. [20] The choice was made to keep the landmark information stored rather than use the RGB pixel information due to the landmark processing being faster and less computationally heavy for the Jetson. This was a key design choice that reduced the latency of sign language characterisation.

Characterisation of the landmark coordinates was achieved using a Support Vector Machine (SVM) to categorise each frame into a letter with a minimum confidence threshold. The SVM chosen was created by Eren Taştepe from the Brunel University as a master thesis project. [21] This was the only model for BSL that was found that was also compatible with the MediaPipe landmark coordinates, which was a key part of the framework. This was the largest limiting factor as the SVM was trained on a single person, signing numbers 0-10 and the alphabet. This may have reduced the accuracy of the model, thus contributing to the miscategorisation of some symbols.

Another limiting factor of this model was the lack of dynamic sign support, as the model's input is individual frames. An attempt was made to identify the start position and track key fingers to identify letters H and J, but was ultimately too unreliable and was removed from the final version of the project.

D. Kinesics: Arm and Upper Body Control

As part of our goal to assist travellers, we designed the robot's communication system to be as inclusive and informative as possible. As part of this, we used Pepper's head, arms, and upper body motions to transmit non-verbal communication cues to complement verbal interaction.

As a first step to allow integration with the wider system, we focused on implementing a small set of simple, clearly distinguishable deictic gestures to highlight directionality in guidance. These gestures were created using kinaesthetic teaching, where Pepper's arms were physically guided through

the desired motions and recorded directly as ROS bag trajectories. This allowed us to capture natural and intuitive movements without manually specifying joint configurations, while also producing reusable motion primitives. The resulting set includes both deictic gestures (e.g. pointing left/right) and interaction-specific gestures such as “follow me”, “sorry, I didn’t hear that”, and a looping “thinking” gesture. Together, these form a strong basis upon which more expressive communication can be built.

These recorded trajectories are replayed within a dedicated ROS2 node that coordinates speech and gesture output. The node subscribes to high-level interaction commands and maps key words, phrases, or interaction states to the appropriate gesture. For example, directional phrases such as “eastbound” or “left” trigger pointing gestures, while system states such as “thinking” activate a looping background animation. This provides a simple and effective mechanism for linking linguistic content to physical motion without requiring complex real-time planning.

A key aspect of the final implementation is the temporal coordination between speech and gesture. Rather than triggering gestures at fixed times, we estimate when a relevant phrase will be spoken and align the gesture accordingly. This is achieved by approximating speech duration using a words-per-second model, combined with a small startup offset to account for speech latency. By identifying a linguistic anchor (e.g. the word “left”), we infer the appropriate delay before triggering the gesture, resulting in synchronised speech and motion that is more intuitive for the user.

Finally, the system supports both discrete and continuous gesture behaviours. Some gestures, such as pointing or “follow me”, are executed once at a specific moment in the interaction, while others, such as the “thinking” gesture, are looped continuously until the system state changes. This allows Pepper to convey both directional information and interaction state, improving clarity and user understanding. Overall, this approach provides a lightweight but extensible framework for combining speech with physically grounded gestures, forming a foundation for more expressive multimodal interaction in future work.

E. User Interface

To support intuitive human–robot interaction in a public transport environment, a clear and accessible user interface was implemented. Train stations present a challenging interaction context, as users may differ widely in language proficiency, hearing ability, and familiarity with robotic systems. The interface was therefore designed to minimise ambiguity and provide clear interaction options when a user first approaches the robot.

When interaction begins, the tablet interface presents available communication methods and languages. Visual language selection flags allow users to quickly choose their preferred language without requiring prior verbal interaction. This approach is particularly important in international transport hubs where passengers may not share a common spoken language. Providing a recognisable and visually intuitive language selection mechanism helps reduce hesitation and allows users to begin interacting with the robot immediately.

Once interaction has started, real-time speech transcription displays what the robot has interpreted from the user’s speech. This feedback enables users to identify and correct recognition errors, increasing transparency and reliability during spoken interaction. In addition, responses generated by Gemini [11] are displayed as text on the tablet interface. This ensures that users who are deaf or hard-of-hearing can still receive guidance, while also supporting communication in noisy station environments where spoken responses may be difficult to hear.

To further improve accessibility, the interface includes a dedicated British Sign Language (BSL) interaction mode. When selected, users are presented with a menu of commonly requested London Underground stations represented as large, clearly labelled buttons. This allows users to quickly select a destination without requiring complex input, reducing interaction time for frequently requested routes.

If the desired destination is not listed, users can enter a custom station name through a sign-based spelling interface. In this mode, the system detects individual BSL alphabet signs using the vision system and combines them to form the station name. A visual status area indicates the current interaction state (i.e. “Waiting for letters”), providing clear feedback that the system is actively monitoring for input.

This two-stage design—quick selection followed by sign-based spelling—balances efficiency and flexibility. Users are able to select common destinations immediately, while still retaining the ability to specify any station when necessary. Throughout the interface, large buttons, clear visual prompts, and consistent feedback indicators were used to reduce cognitive load and support first-time users.

Finally, visual processing indicators are displayed while the system generates responses. These indicators signal that the robot is actively handling a request, helping manage user expectations during processing delays and preventing users from assuming that the system has stopped responding.

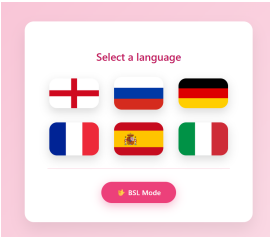

Fig. 5. UI: home page

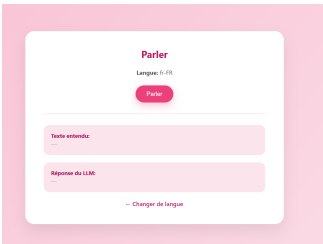

Fig. 6. UI: language example

F. Path Planning and Collision Avoidance

The path planning and collision avoidance module enables the robot to autonomously navigate from an initial position to a specified goal while minimising travel distance. The overall navigation strategy was based on a global path-planning

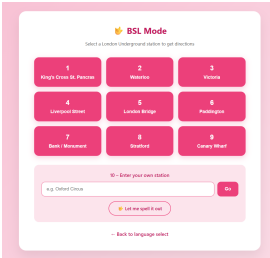

Fig. 7. UI: quick select in BSL

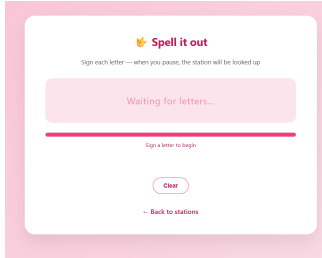

Fig. 8. UI: Spell a station

approach using the A* algorithm [22]. Establishing reliable collision avoidance provided a safety-critical foundation upon which the global A* planner was integrated, allowing for efficient route planning whilst maintaining robustness in the presence of dynamic or partially known obstacles.

1) Collision Avoidance: Progress was focused on the development of a local collision avoidance system with the intent to act as its aid. The native obstacle avoidance system on the Pepper utilises a combination of the sonar and lasers, but samples at low intervals and has a limited range of detection, as shown in Figures 9 and 10 [23].

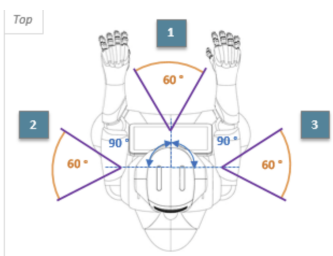

Fig. 9. Pepper's Laser Range of Detection

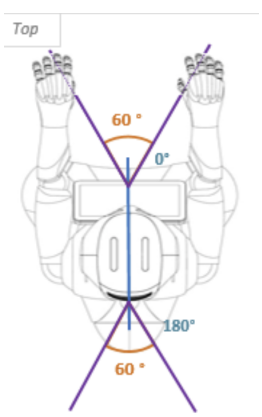

Fig. 10. Pepper's Sonar Range of Detection

Pepper's infrared, sonar, and laser sensors have been tested to detect nearby obstacles, but have proved to be quite inaccurate and to only sample at infrequent intervals. For example, the plot in Figure 11 shows the data collected from the laser scanner in the Pepper robot, which was provided with some short-distance obstacles. As seen, the laser has a low sample rate, and was unable to clearly differentiate between obstacles.

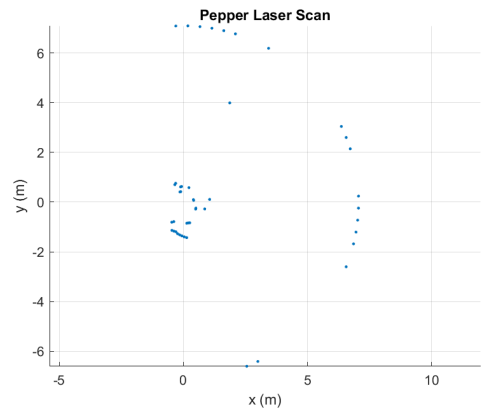

Fig. 11. Laser scan from Pepper (centred at (0,0)) with short-distance obstacles

Following this, a more reliable and robust form of data collection was selected - the RPLIDAR A2M12 [24]. This module allows the Pepper to collect spacial data with a much higher sampling frequency and density. It has a much longer range of detection (12m compared to the previous 5m), and has a larger radius if detection, of 360°.

The LiDAR works to track moving obstacles and estimate where they will be in the next 3 seconds to see if they'll make contact with Pepper. The way it does this is by grouping together nearby points using DBSCAN algorithms to differentiate between moving obstacles [25]. These moving obstacles are then tracked over several scans to calculate their velocity and position relative to the lidar. This position and velocity data is then used to estimate the trajectory of the obstacles over the next few seconds, and to see if they will collide with Pepper. To reduce the complexity of the calculations, the dot product is used to determine if the cluster of data points is heading towards the lidar or not. If they are not, these groups of points are then ignored, thereby decreasing the computational load on the Raspberry Pi. Safe locations for the robot to go to are then calculated to be at 45° or 90° to the current heading, to enable the robot to smoothly avoid oncoming obstacles without straying too far from the intended path.

Seeing as all velocity is calculated relative to the LiDAR, this obstacle avoidance works on both moving and stationary obstacles. The thresholds that informed the avoidance model were chosen iteratively based on real-world testing. These thresholds included the minimum incoming obstacle velocity required to initiate the avoidance, constant distance of the Pepper to the lidar (to ensure the avoidance system ignored the constant detections of the robot), and time of prediction interval of the obstacles future location, among others.

2) Mapping: An isolated environment employing the A2M12 LiDAR, the *ros2_laser_scan_matcher* library [26], and the Canonical Scan Matcher (CSM) [27] alongside SLAM Toolbox [28] was used to construct an occupancy grid map. Drawing inspiration from similar approaches in handheld LiDAR mapping [29], the system determines its pose within the environment using consecutive scans from the LiDAR, a process known as Scan Matching which produces odometry, circumventing the need for wheel encoder data. This estimated motion is published to SLAM Toolbox via the /tf and /odom topics, which incrementally builds the occupancy grid. Examples of the resulting maps are shown in Fig. 14, comprising a raw map and a cleaned version processed using the GyroPalm web-based map editor [30].

Fig. 12. (a) Lab 5 raw map

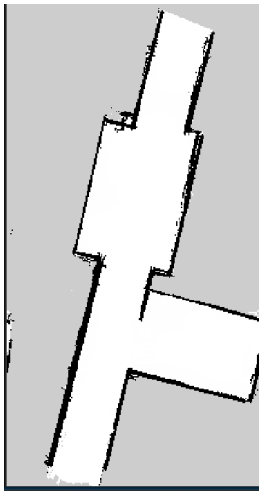

Fig. 13. (b) Floor 10 corridor cleaned map

Fig. 14. Occupancy grid maps generated by the pipeline. Left: raw output from SLAM Toolbox. Right: cleaned version with improved wall definition, processed via GyroPalm [30].

The motivation for operating in an isolated environment stems from the poor odometry quality of the host platform (Pepper), whose wheel encoder data is susceptible to cumulative drift. In practice, this drift caused the map to rotate in alignment with the robot upon any change in orientation, producing overlaid structures that rendered the output unusable. By decoupling the mapping system from the Pepper platform entirely and deriving odometry from LiDAR scans, this source of error is eliminated.

The implementation consisted of five simultaneous processes across independent terminals, with an additional terminal used upon completion to serialise the map to disk [31]. Reliable pose estimation was achieved by using odometry derived directly from consecutive LiDAR scans, using the *ros2_laser_scan_matcher* ROS2 node [26]. This node wraps the CSM library [27], which computes the rigid-body transformation between successive scan frames by solving the point-to-line Iterative Closest Point (ICP) problem [27], yielding a continuous estimate of pose and position within the environment. A static transform was additionally broadcast between the base_link and laser frames to establish the sensor's fixed spatial relationship to the platform, ensuring geometric consistency throughout the session. A custom SLAM Toolbox parameter file was also required to align the TF tree with the expected topic conventions [28]. All active processes were visualised in real time using Foxglove Studio [32] via the Foxglove Bridge. Together, these processes produced the maps shown in Fig. 14, which serve as the environmental representation underpinning the navigation stack.

G. Mobility and Motor Control

This functionality is separated into its own submodule to ensure that safety-critical collision avoidance remains responsive, even when global route planning is incomplete or unavailable. Integrating the *naoqi driver2* package enables the Pepper robot to perform advanced teleoperation and autonomous navigation within the complex environment of a tube station. By bridging the robot's internal Naoqi OS with the ROS 2 Humble framework, the system leverages professional-grade navigation stacks to move beyond simple manual control.

The robot utilises a pre-loaded occupancy grid map to plan efficient paths while simultaneously processing real-time data from onboard lasers, sonars, and the Femto Bolt camera to detect and avoid dynamic obstacles such as commuters. This combination of global mapping and local sensory feedback allows the robot to navigate narrow corridors and busy platforms reliably, making it suitable for mobile assistance and facility monitoring.

The motor control system was developed through a five-stage iterative process, beginning with manual teleoperation using a WASD interface. This stage validated the robot's responsiveness and collision avoidance capabilities. A key feature introduced was the "Return to Origin" function, enabling the robot to autonomously navigate back to a reference point, such as a charging station.

To improve user interaction, the robot performs a final rotation to face toward nearby people rather than stopping facing a wall or dock. Autonomous navigation was further refined to handle obstacles such as walls, gates, and restricted areas. Two re-routing strategies were evaluated: the Rotate-Scan-Commit method and the 8-Direction Commit method. While the former provides a thorough $360°$ scan, it introduces significant delay ($\sim 17\,\mathrm{s}$). The latter proved more efficient, allowing quicker path recovery with minimal computational overhead.

PID controller tuning presented challenges, particularly in balancing settling time and overshoot through the derivative term K_d. High K_d values caused excessive momentum and overshoot, while low values led to unstable "swaying" and poor final orientation.

Wheel odometry was developed as a fallback mechanism to maintain basic localisation in the event that primary odometry or sensor fusion methods fail. However, wheel slippage on uneven surfaces introduces cumulative errors, causing drift between the robot's estimated and actual position. Future work

will focus on improving primary localisation methods and integrating robust sensor fusion to minimise reliance on this fallback approach.

Nav2 exposes plugin interfaces covering costmap layers, planners, controllers, behaviour trees, and behaviours, allowing the robot to navigate autonomously through busy transport environments such as tube stations. Navigation algorithms are implemented as plugins running on ROS action servers, where each server can host one or more algorithm plugins each configured to suit Pepper's specific hardware and the crowd-dense, dynamic spaces typical of TfL infrastructure.

H. Mounting and 3D printing

The additional hardware components were strategically mounted on to the Pepper robot using custom 3D printed fixtures. The Orbbec Femto Bolt Camera and the ReSpeaker microphone were positioned on the robot's head to provide a large field of view for human detection, and so that the microphone was as close to face level as possible to capture speech clearly. To facilitate obstacle avoidance, the RPLIDAR A2M12 was mounted at the base to effectively capture objects and features on the floor for the Pepper to avoid. The Jetson, Raspberry Pi and power bank were housed in a custom backpack, so that these bulkier items would not impede the mobility of the TFLBot. This distributed mounting strategy allowed the system to integrate advanced perception and processing while maintaining the robot's physical balance and mobility.

V. Evaluation Methodology

To assess the impact of mobility and enhanced communication abilities on user experience, we conducted a comparative user study in a controlled underground-station-like environment. In our case, we used the Electrical and Electronic Engineering building as a safe, controlled proxy with classrooms acting as platforms. Users are asked to navigate predefined routes, such as moving from the 10th floor landing to room 1004, under one of the following conditions:

1) no guidance, where the user navigates independently using environmental signage
2) a stationary robot providing verbal directions only
3) a mobile robot which physically guides the user to the room, providing visual indicators for support

The results for the same routes will be compared using the following metrics:

Performance metrics

- *Task completion time:* time taken to reach the target destination

User experience metrics

- *User cognitive load:* measured using adapted post-task NASA Task Load Index survey [33]
- *User satisfaction and perceived usefulness:* collected via Likert-scale survey questions

Participants were randomly assigned to each condition to minimise bias. All guidance systems were configured to provide equivalent route information, ensuring that performance differences arise primarily from the mode of interaction rather than content differences.

External factors such as crowding and time of day were controlled during the study to minimise influence on metrics such as cognitive load and user satisfaction.

VI. Results and Discussion

Test	Time taken (s)	Load (/5)	Satisfaction (/5)
No Aid	25.2	2.0	–
Static Aid	20.4	1.6	3.32
Mobile Aid	81.4	1.4	3.68

TABLE I
Summary of measured metrics.

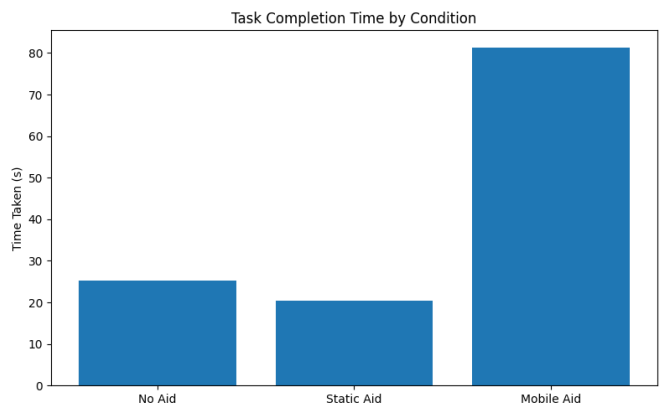

Fig. 15. Mean task completion time by condition.

Table I summarises the average results from five trials under each test condition. To complement these aggregate values, Figures 15 and 16 shows the comparison of cognitive load and task completion time across conditions, while Figure 17 presents the subjective comparison between the stationary and mobile robot conditions. During participant selection, we aimed for a diverse quorum so that the collected data would better reflect a broader user group. The results partially support Hypothesis H1. The introduction of the TFLBot significantly reduced cognitive load and improved user satisfaction compared with the no-aid condition. As shown in Table I, cognitive load decreased from 2.0 in the no-aid condition to 1.55 and 1.4 with the static and mobile aids, respectively. This trend is also visible in Figure **??**. In terms of efficiency, the stationary TFLBot reduced task completion time from 25.2 s (no aid) to 20.4 s, demonstrating that even simple verbal guidance can improve navigation efficiency in a structured environment.

However, the mobile TFLBot did not reduce navigation time relative to the baseline. Instead, the average completion time increased substantially to 81.4 s, compared with 25.2 s without assistance and 20.4 s with the static aid. This was due in part to

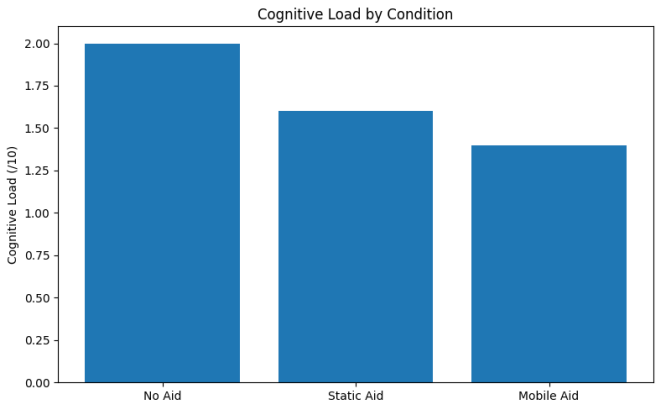

Fig. 16. Mean cognitive load by condition.

the robot's conservative navigation behaviour, including a low base velocity and frequent stopping for collision avoidance. Each detected obstacle caused the robot to pause and recompute a safe path, resulting in repeated interruptions during navigation.

In addition, one mobile-robot trial was treated as an outlier in interpretation because the robot misheard the requested station and navigated to the wrong platform. This significantly inflated the recorded completion time for the mobile condition and does not reflect the intended guidance behaviour. We therefore report this as a system failure case rather than representative performance.

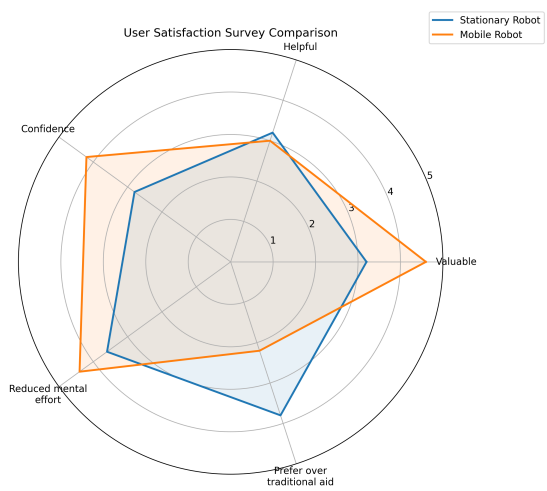

Fig. 17. Radar plot comparing subjective satisfaction ratings for the stationary and mobile robot conditions.

Despite the increase in completion time, the mobile TFLBot achieved the highest satisfaction score of 3.68, compared with 3.32 for the static aid, indicating an improvement in perceived user experience. The results therefore support H1 in terms of workload reduction and user satisfaction, but only partially support it in terms of navigation efficiency.

The results do not fully support Hypothesis H2. While the mobile robot was expected to outperform the stationary system across all metrics, it instead introduced delays associated with movement and navigation. Nevertheless, Figure 17 shows that the mobile robot was rated more highly than the stationary robot in perceived value, confidence, and reduced mental effort. This aligns with the satisfaction scores reported in Table I, where the mobile condition achieved the highest rating.

It can also be seen that the static TFLBot reduced cognitive load from 2.0 to 1.55, while the mobile TFLBot reduced this level further to 1.4. Participant feedback indicated that a significant portion of the remaining cognitive load was associated with interacting with the system itself, particularly uncertainty over whether speech input had been successfully recognised. Several participants reported needing to speak more clearly or more loudly, and suggested that explicit feedback indicating active listening would improve usability. In contrast, a human guide naturally communicated understanding through body language and other non-verbal cues.

User satisfaction increased with the level of interactivity provided. While the no-aid condition produced low satisfaction (not recorded numerically in Table I), both robotic conditions improved user experience, with the mobile TFLBot achieving the highest satisfaction score of 3.68. Although the TFLBot did not reach the performance of a human guide, it demonstrated a clear and meaningful improvement over no assistance, particularly in terms of reducing cognitive burden and improving the overall user experience.

VII. Future Work

The current state of the TFLBot is able to provide directions to users in need there by meeting it primary directive. It also served well against the hypothesis as to a user in need absence of aid is a demerit and as a guide to platforms the user following directions themselves proved to be more efficient than the it leading the way, this came down to the speed at which it could complete its work flow from initial contact to arrival at the platform. This presents an opportunity for growth among other aspects of the project.

A. Further Implementation

Following the completion of the project, we reviewed limitations in the current system and identified areas for extended work. These include developing a purpose-built robot for underground travel assistance, improving the user interface with additional real-time service updates, expanding language support, improving hand sign recognition accuracy, and increasing overall mobility and navigation efficiency.

1) Specialised Robot: Using Pepper highlighted several limitations, particularly in environmental data collection and overall system capability. This introduced constraints and reinforced the need for a purpose-built platform. A specialised robot could incorporate improved IMU data processing, higher quality microphones, cameras, and integrated IR, sonar, and laser sensors within a more user-friendly design, rather than

relying on external additions. It would also enable greater mobility across complex underground environments, including navigation via elevators, escalators, and stairs.

2) Live Feed of Train Updates: The current user interface could be enhanced to better support both users and operators. Integrating real-time updates on train line status would allow for more effective crowd redirection in multiple languages. Additionally, incorporating an emergency communication link to underground staff could reduce response times and improve overall system usefulness in critical situations.

3) Versatility with Languages: There is significant scope to improve multilingual interaction. Advances in real-time translation, such as those seen in consumer technologies like Samsung's live translation earbuds, highlight the potential for more natural and accurate communication. Expanding language support, including better handling of accents, would improve accessibility and overall user interaction.

4) More Adept Model Training for Sign Language: While the final system could recognise numbers and individual letters, it struggled with conversational sign language, largely due to limited training data. Sign language relies heavily on context and facial expression, so expanding the dataset to include a wider range of conversational signs would improve accuracy and reliability. This would significantly enhance inclusivity and user experience.

5) Efficiency in Mobility: The robot's ability to guide users was less efficient than intended. Improving internal communication and response times would make navigation smoother and more reliable. This would also broaden accessibility, particularly for users who may struggle with language barriers, allowing the system to better fulfil its intended role.

VIII. Conclusion

This design report presents the system architecture and evaluation methodology for the TFLBot, a mobile robotic guide for navigating public transport networks. The system integrated a SoftBank Pepper robot with enhanced computation and sensing, including a Raspberry Pi 5, an NVIDIA Jetson Orin Nano, an Orbbec Femto Bolt camera, a microphone array, and an RPLIDAR A2M12. This report also details our research hypothesis and a review of relevant background literature that informed the system design and component selection.

References

[1] G. Robotics, "Pepper robot deployed in train stations by sncf," https://www.gwsrobotics.com/robots-airports-stations, 2015, pepper was deployed in three train stations for passenger information tasks between December 2015 and March 2016.

[2] LG Electronics, "Lg introduces airport guide robot," https://www.lgcorp.com/media/release/7871, 2017.

[3] ——, "Lg cloi guidebot: Indoor service robot," https://www.lg.com/global/business/service-robots, 2019.

[4] K. O. Arras, D. Cerqui *et al.*, "Spencer: A socially aware service robot for passenger guidance and help in busy airports," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 8, pp. 2541–2553, 2018.

[5] Roboteam Home, "temi: The personal robot," https://www.robotemi.com, 2020.

[6] Enchanted Tools, "Mirokaï: A social robot designed for public spaces," https://enchanted.tools/robot, 2023.

[7] SoftBank Robotics Europe, "Pepper as a service and concierge robot," https://www.softbankrobotics.com/emea/en/pepper, 2019.

[8] N. J. Hetherington and E. A. Croft, "Nonverbal motion legibility cues for human–robot spatial interaction," *arXiv preprint arXiv:2104.02275*, 2021.

[9] *Raspberry Pi Documentation*, Raspberry Pi Foundation, 2024. [Online]. Available: https://www.raspberrypi.com/documentation/

[10] *Jetson Orin Nano Developer Kit User Guide*, NVIDIA, 2023. [Online]. Available: https://developer.nvidia.com/embedded/learn/jetson-orin-nano-devkit-user-guide

[11] Google, "Gemini api documentation," 2024, accessed: 28-Jan-2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[12] *User Guide for :contentReference[oaicite:0]index=0*, Anker, 2025. [Online]. Available: https://support.anker.com/s/article/Anker-Power-Bank-25K-165W-Built-in-and-Retractable-Cables-User-Guide-A1695

[13] GL-iNEt, "Gl-ar750 travel ac router," Tech. Rep., 2024. [Online]. Available: https://www.gl-inet.com/products/gl-ar750/

[14] V. Rabaud, K. Knese, and M. Arguedas, "naoqi_driver2: Ros2 driver for nao and pepper robots," GitHub Repository, Tech. Rep., 2024. [Online]. Available: https://github.com/ros-naoqi/naoqi_driver2

[15] Orbecc, "Python orbecc sdk," GitHub Repository, Tech. Rep.

[16] D. Mishra, G. A. Romero, A. Pande, B. N. Bhuthegowda, D. Chaskopoulos, and B. Shrestha, "An exploration of the pepper robot's capabilities: Unveiling its potential," Norwegian University of Science and Technology (NTNU), Tech. Rep., 2024.

[17] Seeed Studio, "Respeaker usb mic array," https://wiki.seeedstudio.com/ReSpeaker-USB-Mic-Array/, 2024, accessed: 28-Jan-2026.

[18] O. D. T. International, "Orbbec femto bolt hardware specifications and robotics applications," Orbbec, Tech. Rep., 2024. [Online]. Available: https://www.orbbec.com/products/tof-camera/femto-bolt/

[19] M. P. Michalowski, S. Sabanovic, and R. Simmons, *A Spatial Model of Engagement for a Social Robot*, IEEE, Istanbul, Turkey, 2006.

[20] *MediaPipe Solutions guide*, Google. [Online]. Available: ttps://ai.google.dev/edge/mediapipe/solutions/guide

[21] E. Taştepe, "Bsl svm model." [Online]. Available: https://www.kaggle.com/datasets/erentatepe/bsl-numbers-and-alphabet-hand-position-for-mediapipe

[22] Z. Lin, K. Wu, R. Shen, X. Yu, and S. Huang, "An efficient and accurate a-star algorithm for autonomous vehicle path planning," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 6, pp. 9003–9008, 2024.

[23] S. Robotics, "Pepper - documentation," accessed: 10-March-2026. [Online]. Available: http://doc.aldebaran.com/2-5/family/pepper_technical/laser_pep.html

[24] DFRobot, "Rplidar a2m12 - 360 degree laser scanner development kit," https://www.dfrobot.com/product-1461.html?srsltid=AfmBOoocKvWrDqg5d6-knk9QLrn-lsKAQfeotp3GEpJwpKlr-s92gGCU, 2024, accessed: 10-Feb-2026.

[25] Scikit-Learn, "Dbscan," accessed: 12-March-2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html

[26] A. Karavaev, "ros2_laser_scan_matcher," accessed: 19-March-2026. [Online]. Available: https://github.com/AlexKaravaev/ros2_laser_scan_matcher

[27] A. Censi, "An icp variant using a point-to-line metric," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2008, pp. 19–25, accessed: 19-March-2026. [Online]. Available: https://github.com/AndreaCensi/csm

[28] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Slam toolbox: Slam for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021, accessed: 19-March-2026. [Online]. Available: https://github.com/SteveMacenski/slam_toolbox

[29] T. Shan and B. Englot, "Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 4758–4765, accessed: 19-March-2026.

[30] GyroPalm, "Online map cleaning tool," accessed: 19-March-2026. [Online]. Available: https://gyropalm.com

[31] O. N. LLC, "Navigation2 (nav2) documentation," accessed: 19-March-2026. [Online]. Available: https://docs.nav2.org

[32] F. Technologies, "Foxglove studio documentation," accessed: 19-March-2026. [Online]. Available: https://docs.foxglove.dev

[33] NASA, "Nasa task load index," https://humansystems.arc.nasa.gov/groups/tlx/downloads/TLXScale.pdf, 1986.